# Repeating Efficiently

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Helpful Classroom Sorter

---

TEACHER GUIDE · 1 OF 9

## Repeating Efficiently

> Prepare Repeating Efficiently: check the target, prerequisites, resources and adaptable teaching schedule.

### Teach the learning cycle

Primary target: Trace a **loop** by testing its condition before every repetition

Session · Sequence and minutes · Resources

> **Session:** Session 1 · Understand · 45 min
>
> **Sequence and minutes:** 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.
>
> **Resources:** Use Your mission, Notice the evidence, See how it works, Compare the cases, Words that unlock the idea and Follow a worked example; record the brief recall in the workbook.

> **Session:** Session 2 · Practise · 45 min
>
> **Sequence and minutes:** 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.

> **Resources:** Use Find and repair a mistake, Practise together, Run the investigation and Build your understanding; use the matching workbook W2 and misconception check.

> **Session:** Session 3 · Investigate and transfer · 45 min
>
> **Sequence and minutes:** 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.
>
> **Resources:** Use Run the investigation, Connect your project, Show what you know, Study a complete case and Try a new case independently. Allow extra time for the full task set. Use a coding lab only when this week actually provides one; Your lesson route is the planning section.

This is a starting allocation of 135 minutes, not a validated duration. Preserve all named topics. Schedule extra time for connected examples, the full investigation, independent cases and project building.

Repeated copies of a case share one recorded attempt; they are not new **evidence**.

### Retrieval answer guidance

### If/Then Decisions

Without opening the old lesson, explain one key idea from If/Then Decisions. Give an example and a mistake that the idea helps you avoid.

Look for: Evaluate a condition using the current state and trace only the selected branch

Reference example, not the only acceptable wording: Start 2: after adding 2 score is 4; at least 6 is false; add 4 and report 8. Start 5: after adding 2 score is 7; at least 6 is true; subtract 3 and report 4.

Each run starts fresh and executes only its selected branch.

### Features: Useful Clues

Without opening the old lesson, explain one key idea from Features: Useful Clues. Give an example and a mistake that the idea helps you avoid.

Look for: Choose recorded properties that preserve distinctions needed for a stated task

Reference example, not the only acceptable wording: Use length and width measured before the decision. State the order as length in cm, then width in cm; the record is 12, 8.

The target is fits or too large under the guide, and this card is fits because both equalities are included.

Colour and card number are not needed for this rule; the final inspection sticker copies the answer and is unavailable before inspection.

### Pictures Become Numbers

Without opening the old lesson, explain one key idea from Pictures Become Numbers. Give an example and a mistake that the idea helps you avoid.

Look for: Encode and reconstruct a small image using an explicit grid convention

Reference example, not the only acceptable wording: The upper row has black, white, black; the lower row has white, black, white. Reading back gives the supplied two rows.

There are three black pixels among six total positions.

Use the student module and workbook section with this exact week title. The worked case, independent question and answers below form one aligned lesson sequence.

### What the learner must demonstrate

A robot stamps the next unstamped card until it has stamped five cards. Only three cards are available. Write a stopping rule that handles both limits and trace the number stamped.

### Concept to explain

A **loop** repeats a useful step until a **stopping condition** is met.

| Term | Meaning |
| --- | --- |
| loop | control structure that repeats steps |
| iteration | one pass through the loop |
| loop body | steps repeated each time |
| stopping condition | rule that ends repetition |

### Prepare the class

Use the supplied scenario, student illustrations and blank paper. For numeric work, offer counters or a calculator after learners show the setup. Fictional records are supplied; personal data and paid accounts are unnecessary.

#### Alternative compact allocation

> This compact option is not the full-week schedule. Use the main teaching cycle and allow extra sessions for the investigation, independent work and project.

Allow 80 minutes: recall 8; explanation 12; worked demonstration 15; guided practice 20; independent assessment 15; project and reflection 10. Extend the lab into a second session when needed.

### Preparation for the connected explanation

Teach a pretest while **loop** with exact whole-number arithmetic. Track the instruction pointer, current state, test result and **body** count separately. Use both zero-**iteration** and boundary-crossing cases.

The termination explanations depend on the stated update and condition; do not claim that every increasing counter terminates. Faulty **loop**s are paper traces, stopped by the teacher once the reason for repetition is clear.

### Readiness answer

The test is true, so add 3 and report 7. This single decision does not repeat. Another test must be explicitly specified.

A counter card, a while condition card and a separate **body** card with an arrow back to the condition

A trace sheet with current value, true or false and action columns, plus a separate **iteration** tally

Starting-value cards including 1, 6, 7 and 8, and an add 0 fault card

### End goal for the pictorial case

By the end, I can check a **loop** condition before each update and count **iteration**s.

This added worked case illustrates the week's main idea. Retain all original and connected topics; schedule its practice within the teaching cycle.

---

TEACHER GUIDE · 2 OF 9

## Explain the mechanism

> Model Repeating Efficiently using the supplied case, labelled mechanism and reasoning. Make the scope of each conclusion explicit.

Follow the information instead of imagining magic inside the machine.

**1. Input — information entering**

**INPUT** starting state and repeat count

**2. Process — the deciding step**

**PROCESS** run the **loop body** and update **progress**

**3. Output — the result**

**OUTPUT** finished pattern when the stop condition is met

### Key idea

CORE IDEA A **loop** repeats a useful step until a **stopping condition** is met.

### How the mechanism works

UNDER THE HOOD For these bounded counter exercises, specify the starting value, repeated action, update and stopping test. Check that the updates eventually make the test false; otherwise a run may continue forever or repeat the wrong number of times. Other programs may use different **loop** designs, such as waiting for a stop signal.

Draw a fresh example with all three stages:

Every step connects a claim to observable **evidence**.

### The case to investigate

CASE Instead of writing draw-side and turn four times, a square algorithm can repeat those steps four times.

### Step 1

STEP 1 — OBSERVE starting state and repeat count

### Step 2

STEP 2 — TRACE run the **loop body** and update **progress**

### Step 3

STEP 3 — CONCLUDE finished pattern when the stop condition is met

### Model answer

MODEL REASONING Place only the repeated steps inside the **loop**, initialize the counter, update it once per pass, and stop after the required count. Trace each **iteration** to catch one-too-many or one-too-few errors.

### A claim to avoid

DO NOT OVERCLAIM Repeat forever is not an efficient algorithm unless endless operation is truly the goal and can be stopped safely.

What new **evidence** could change this answer?

**Concrete teaching case**

### The teaching case

Repeat take the next unstamped card and stamp it four times.

How many cards are stamped? What assumption matters?

**Exact explanation to model**

### Why the result follows

Four, provided four unstamped cards exist and each repeat takes the next one. A limit or end-of-stack check prevents continuing indefinitely.

### Explain the added worked example

A paper counter starts at 1. While the counter is below 7, add 2 to it. When the test is false, report the counter and stop.

The word while means test first, run the **body** only if true, then return to the test using the updated value. The **body** here is just add 2.

The counter follows 1, 3, 5, 7. At 1, 3 and 5 the condition is true, so each test is followed by one addition. At 7 the condition is false. No addition follows that final test. The **loop** has three **iteration**s and reports 7.

## Explain the added worked example · 1

Compare the Week 10 instruction if counter is below 7, add 2, then report. From 1 that single decision reports 3. A while **loop** explicitly returns to its test; an if decision does not.

Draw a pointer to the instruction being executed instead of deciding for yourself when to repeat.

The trace has four tests but only three **iteration**s. The false test is **evidence** that this run stops. A list ending at 5 would be incomplete, while a list continuing to 9 would include an addition the **loop** did not authorise.

Write the current value, the test result and the action on every row. Repeated values can matter: if the **body** does not change the state used by the test, the same true test may occur again.

Do not erase these rows simply because they look unhelpful.

### Key terms

| Term | Meaning |
|---|---|
| Loop | Instructions that repeat according to a stated rule |
| Iteration | One execution of the repeated body |
| Body | The instructions performed on each iteration |
| Stopping condition | The situation in which repetition ends |
| Progress | A change that brings the state closer to a stopping condition |

### Model the reasoning aloud

Use the complete worked example below before asking learners to complete W2. Reveal a small part, invite a **prediction**, then explain the deciding step.

These teacher notes contain solutions; do not reveal the independent case answer during modelling.

| Starting counter | Values at successive tests | Iterations and reported value |
|---|---|---|
| 5 | 5 true then 7 false | 1 iteration and report 7 |
| 6 | 6 true then 8 false | 1 iteration and report 8 |
| 7 | 7 false | 0 iterations and report 7 |
| 8 | 8 false | 0 iterations and report 8 |

While tests before the **body**. If the first test is false, the **body** runs zero times. Do not add 2 once just because a **loop** is present. Starting at 7 or 8 is a useful test of this rule.

A **loop** that stops when the counter is no longer below 7 does not promise to finish at exactly 7. Starting at 6 produces 8. Passing the boundary is allowed by these instructions.

Always report the actual final state instead of replacing it with the threshold.

| Thinking move | Teacher prompt |
|---|---|
| Plan | What is given? What is the question? Which procedure fits these facts? |
| Do | Point to each input and intermediate step in the worked trace. Name the rule that allows the next step. |
| Check | Does the output answer the question? Which assumption or limitation prevents a larger claim? |

Use the diagram and verbal explanation together. Ask learners to match a sentence to a specific visual step; do not assign learners fixed visual or auditory learning styles.

### A pictorial worked case: Repeating Efficiently

A fictional paper counter advances through a display until its condition becomes false.

## The facts we will use

### Supplied facts

Start at 2. While the current counter is below 10, add 3.

### Supplied facts

Test before each addition; report when the condition is false.

AI ACADEMY / AI-1/11

## Condition checks before additions

| | Current | Below 10? | Next |
|---|---|---|---|
| Check 1 | 2 | true | 5 |
| Check 2 | 5 | true | 8 |
| Check 3 | 8 | true | 11 |
| Check 4 | 11 | false | stop |

Read a cell using both its row and column labels.

Follow the labelled picture with the matching explanation. Open the vector diagram

AI ACADEMY / AI-1/11

# Repeating Efficiently

**Start with the supplied facts**

Follow each arrow. Ask which fact allows the next step.



### 1. First test

2<10 is true: update to 5.



### 2. Repeat

5<10: update to 8. 8<10: update to 11.



### 3. Stop test

11<10 is false; do not add again.



### 4. Report

Final counter 11 after three additions.

**CHANGE ONE THING**

Start at 10 instead of 2.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

### Read the picture one step at a time

| STEP | WHAT HAPPENS AND WHY |
| --- | --- |
| 1. First test | 2<10 is true: update to 5. |
| 2. Repeat | 5<10: update to 8. 8<10: update to 11. |
| 3. Stop test | 11<10 is false; do not add again. |
| 4. Report | Final counter 11 after three additions. |

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

### Why this result follows

#### Why the result follows

The condition checks whether to do the next update; it does not constrain every resulting value to stay below 10. The final 11 is therefore allowed by this procedure.

Record both the condition and counter to distinguish a correct stop from an assumed endpoint.

### What this example does not establish

#### Keep the stated limit

This **loop** stops because positive increments eventually cross the limit. Other updates can

fail to make **progress**.

**Change one thing in the pictured case**

### Change one thing

Change: Start at 10 instead of 2.

### Predicted result

**Prediction**: The **loop** runs zero times and reports 10.

### Why the result follows

Reason: Its first condition is already false, so no addition happens.

**Ask while showing the picture**

### Ask while showing the picture · 2

Cover the result. Ask learners to name the **input**s and predict the next step. Uncover one step, connect the icon to the actual procedure, and explain its **evidence**. Then change the supplied condition and compare.

The picture is a representation; it does not document execution of a real AI model.

---



## Demonstrate and guide practice

> Demonstrate Repeating Efficiently, reduce support deliberately and check individual reasoning before discussing answers.

## Board sequence

### 1. Put the facts on the board

Write the given facts: Repeat take the next unstamped card and stamp it four times.

### 2. State the decision to explain

Write the question to explain: How many cards are stamped? What assumption matters?

### 3. Reveal the worked reasoning

Reveal after **prediction**s: Four, provided four unstamped cards exist and each repeat takes the next one. A limit or end-of-stack check prevents continuing indefinitely.

### Hold back the independent task

Keep for an independent attempt: Repeat clap then tap three times. Write the complete sequence. How many actions occur? What would happen if the stopping instruction were missing?

## Classroom dialogue

### Ask the class

Ask: Which facts are given, and which would we have to assume? Learners mark relevant details before discussing answers.

### Language to model

Say: Predict first. Show the rule, information path or calculation that would produce the

> result. A correct guess is not yet an explanation.

Reveal reasoning one step at a time. Explain why a tempting alternative fails. Write units and denominators for arithmetic, trace conditions for decisions, and identify supporting sources for claims.

In pairs, one learner solves while the other checks **evidence**; exchange roles. Remove the worked answer before independent assessment.

### Model these guided questions

From 3 under add 2: 3 true, 5 true, 7 false; two **iteration**s and report 7. Under add 0: 3 true repeatedly, with no change and no report. Seeing 3 during that faulty run is not a returned result.

For the exit, start 7 or any larger whole number checks zero **iteration**s and reports that start. Start 6 checks crossing the boundary and reports 8 after one **iteration**.

### Case-specific teaching sequence

| Stage | Teacher action |
|---|---|
| Session 1 start 0 to 7 minutes | Retrieve the single-decision example from Week 10. Establish that reaching a different value does not create repetition without an instruction to return. |
| 7 to 20 minutes | Model start 1, while below 7 add 2. Move a pointer between test and body and include the final false test at 7. Tally only body executions. |
| 20 to 33 minutes | Compare starts 6, 7 and 8. Require students to predict before tracing. Explain overshoot and zero iterations using the written condition. |
| 33 to 45 minutes | Complete W1 and W2 in pairs. Check that learners distinguish four tests from three iterations and never run the body after a false first test. |
| Session 2 start 0 to 8 minutes | Use W3 to retrieve stopping behaviour. Ask students to identify the first extra update before discussing the final answer. |
| 8 to 23 minutes | Complete the guided comparison of add 2 and add 0. Stop the physical demonstration after repeated identical states and ask whether the program itself reached report. |
| 23 to 38 minutes | Collect W4 independently. A decreasing countdown checks transfer beyond the upward counter. Require the final false test on each finite run. |

| Stage | Teacher action |
| --- | --- |
| 38 to 45 minutes | Discuss W5 and the exit tests. Explain why exact equality may be unreachable. Preview finding the first step where expected and observed behaviour differ. |

### Reduce help deliberately

#### Completion practice

Completion task: Use the same below 8 **loop** with add 2. Make separate runs from 8 and 9. Explain how many times the **body** runs and what each run reports. Why would adding 2 once be an error?

#### Reasoning to listen for

Teacher check: Both first tests are false. Each run has zero **iteration**s, reporting its unchanged starting value: 8 or 9. Adding 2 once would execute the **body** despite a false first test.

Start by identifying the given **input**s together. Leave the intermediate steps blank. If needed, model one step, then withdraw that help on the next step. Move to a full trace only when the partial trace is understood.

For partners, alternate explainer and checker; collect a response from every learner before discussion.

### Individual reasoning check

Use the worked case for **Loop**s and stopping rules. Proposed explanation: “Every **loop** runs at least once”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

#### Expected repair

Expected correction: Point to the test before the **body**. From 7 under below 7 the first test is

> false, so no update is permitted.

Collect explanations, not only yes/no votes. If a misunderstanding is common, re-model the relevant step for the class.

If it affects a few learners, give a short targeted group explanation while others solve a more demanding variation.

### Pictorial practice question

> Trace a fresh start of 9. How many additions occur, and must the report equal 10?

### Reasoned teacher answer

9<10 is true, so it becomes 12. Then 12<10 is false: one addition, report 12; equality with 10 is not required.

Changed-case reasoning: The **loop** runs zero times and reports 10. Its first condition is already false, so no addition happens.

Collect each learner's explanation before discussion. Use the first missing connection between **input**, deciding step and result to choose support. This question is worked-case practice, not fresh mastery **evidence**.

---



## Run the weekly evidence lab

> Run the Repeating Efficiently investigation and use the first unsupported step to choose a teaching response.

Predict first, test honestly, and explain what the **evidence** means.

### 1. Predict

State your **prediction** before the result is known.

### 2. Investigate

Complete the task: Compress a repeated drawing algorithm using a **loop** card.

### 3. Compare cases

Keep a normal case and an edge case clearly separated.

### 4. Record honestly

Record what happened—even when it does not match your **prediction**.

### 5. Explain and plan the next test

Explain the result, one limitation, and the next useful test.

### Use a fresh case after support

INDEPENDENCE After support is removed, repeat the proof with a fresh case.

#### Make the lab runnable

Use the concrete case as the shared normal case and the independent question as a second case. Each pair creates a boundary or missing-information case and predicts its result before testing.

Keep the expected result separate from an observed mistake.

| Record | What to capture |
| --- | --- |
| Input or evidence | Exact data, instruction or claim |
| Expected result | Answer or safe fallback declared before the run |
| Observed result | Actual result, including errors |
| Explanation | Rule, calculation or evidence explaining the outcome |
| Next test | One justified change and a new prediction |

### Diagnose and reteach

> **Find the unsupported claim**
>
> CLAIM TO REPAIR Three repetitions always mean three actions.

For guessing, return to given facts and request a trace. For vocabulary without application, use one example and one non-example. For arithmetic errors, check the setup before the calculation.

Finish with a different uncoached case before recording mastery.

### Address these additional misconceptions

Every **loop** runs at least once: Point to the test before the **body**. From 7 under below 7 the first test is false, so no update is permitted.

The final value must equal the threshold: Trace start 6 under add 2 and below 7. The **loop** reports 8 because the stopping requirement is no longer below 7.

Tests and **iteration**s are the same count: Count **body** executions separately and show the last false test with no **body** action.

A changing number guarantees a stop: Compare below 7 with not equal to 7 from 6 under add 2. Even numbers pass 7 without equalling it.

### Match the support to the first error

| What the response shows | Next teaching action |
|---|---|
| Missing or misread facts | Read the case aloud, mark supplied versus unknown facts, and let the learner restate it. |
| Wrong procedure | Return to the deciding step in the worked case. Contrast it with the proposed incorrect explanation. |
| Arithmetic or implementation error | Trace intermediate values or lines; preserve the first mismatch and retry that step. |
| Correct result without a reason | Ask for a trace and a changed case before marking independent understanding. |
| An unsupported wider claim | Ask which supplied evidence supports the claim and which further observation would be needed. |

Offer an oral explanation, drawing, enlarged visual or fewer examples when that removes a reading/writing barrier. Keep the same reasoning goal.

Stretch secure learners by changing a boundary or assumption and asking them to defend the effect, not simply by assigning more of the same questions.

---



## Assess and explain the answers

> Use the Repeating Efficiently questions, teacher keys and criteria after the relevant attempts. Previously discussed cases are practice; protect the separately selected independent task.

### Worked case answer

#### Teacher key · Worked case answer

Four, provided four unstamped cards exist and each repeat takes the next one. A limit or

end-of-stack check prevents continuing indefinitely.

### Independent question

#### Independent question · withhold the key

Repeat clap then tap three times. Write the complete sequence. How many actions occur? What would happen if the stopping instruction were missing?

### Expected independent answer

#### Teacher key · Expected independent answer

Clap, tap, clap, tap, clap, tap: six actions. Without a stopping instruction, the described repetition has no defined end.

### What counts as evidence

| Criterion | 0 points | 1 point | 2 points |
|---|---|---|---|
| Result | Incorrect or absent | Partly correct | Correct with necessary conditions |
| Reasoning | No supporting trace | Some steps | Clear mechanism, calculation or source support |
| Limit | Overclaims certainty | Vague limitation | Relevant uncertainty or missing fact |
| Transfer | Copies without adapting | Adapts with prompts | Solves a changed case independently |

Use 6/8 as a classroom checkpoint, with 2 required for both Result and Reasoning. This is a proposed teaching rule, not a validated certification threshold.

Do not count a coached answer as independent mastery; correct unsafe or unsupported claims regardless of total.

### Feedback

Ask the learner to find the **evidence** that challenges this claim: “Three repetitions always mean three actions.” Keep the first attempt and an uncoached retry separate.

### Original transfer answer

#### Teacher key · Original transfer answer

Continue while stamped<5 AND another unstamped card exists. Counts are 0,1,2,3; stop because the stack is empty. Report three completed and two unavailable.

A **loop** needs a **progress** update and a termination condition; it must not invent missing **input**s.

### Explanations for the additional workbook examples

### W1 Trace every test

#### Workbook question · W1

Start counter at 2. While counter is below 8, add 2. When false, report and stop. Record the value and result at every test, including the final false test. State the number of **iteration**s and the reported value.

#### Teacher reasoning · W1

Tests: 2 true, 4 true, 6 true, 8 false. Three additions are three **iteration**s. The **loop** reports 8; the fourth test does not run the **body**.

### W2 Check the first test

#### Workbook question · W2

Use the same below 8 **loop** with add 2. Make separate runs from 8 and 9. Explain how many

times the **body** runs and what each run reports. Why would adding 2 once be an error?

### Teacher reasoning · W2

Both first tests are false. Each run has zero **iteration**s, reporting its unchanged starting value: 8 or 9. Adding 2 once would execute the **body** despite a false first test.

#### W3 Find the extra repetition

### Workbook question · W3

Start counter at 3. While counter is below 8, add 2. A learner writes values 3, 5, 7, 9, 11 and reports 11. Identify the first unauthorised update. Give the correct final test, reported value and **iteration** count.

### Teacher reasoning · W3

After 7 becomes 9, the next test 9 below 8 is false. The update from 9 to 11 is unauthorised. Report 9 after three **iteration**s, with test rows 3 true, 5 true, 7 true, 9 false.

#### W4 Transfer to a countdown

### Workbook question · W4

A paper countdown starts at 7. While remaining is above 1, subtract 2. When false, report and stop. Trace the run and count **iteration**s. Repeat from 6 and from 1. Explain why the final value need not equal 1.

What happens from 7 if subtract 2 is replaced by subtract 0?

### Teacher reasoning · W4

From 7: 7 true, 5 true, 3 true, 1 false; three **iteration**s, report 1. From 6: 6 true, 4 true, 2 true, 0 false; three **iteration**s, report 0. From 1: first test false, zero **iteration**s, report 1. Above 1 stops at 1 or below.

Subtracting 0 from 7 leaves every test true, so that faulty run never reports.

## W5 Test a proposed stopping rule

### Workbook question · W5

A procedure says start at 4; while counter is not equal to 9, add 2; report and stop. Write the first four values tested. Explain why the **loop** will not reach 9 using whole numbers.

Suggest a changed condition that would stop once the counter reaches or passes 9, and trace that repaired run.

### Teacher reasoning · W5

The first four values are 4, 6, 8, 10. Adding 2 to an even number always gives another even number, so the counter cannot become the odd number 9. Change the condition to while counter is below 9.

Then 4 true, 6 true, 8 true, 10 false; three **iteration**s and report 10.

## Assess independent transfer

### Independent question · withhold the key

Selected case: A paper countdown starts at 7. While remaining is above 1, subtract 2. When false, report and stop. Trace the run and count **iteration**s. Repeat from 6 and from 1. Explain why the final value need not equal 1.

What happens from 7 if subtract 2 is replaced by subtract 0?

### Selected case answer

> **Teacher key · Selected case answer**
>
> From 7: 7 true, 5 true, 3 true, 1 false; three **iteration**s, report 1. From 6: 6 true, 4 true, 2 true, 0 false; three **iteration**s, report 0. From 1: first test false, zero **iteration**s, report 1. Above 1 stops at 1 or below.
>
> Subtracting 0 from 7 leaves every test true, so that faulty run never reports.

| Dimension | Evidence required |
|---|---|
| Explanation | Names the applicable idea or procedure and ties it to the supplied facts. |
| Trace and result | Preserves the given inputs and shows the relevant intermediate steps; distinguishes a prediction from an observed execution. |
| Limits | States the assumptions, missing evidence or conditions under which the result would change. |

> **Teacher key · Selected case answer**
>
> Record each dimension as not yet, with support, or independent. Accept a valid alternative procedure with a defensible trace.
>
> Do not penalise an honestly recorded model failure if the learner correctly explains its **evidence** and limits.

### Feedback that the learner uses

Name the earliest unsupported step and give one action to repair it. Have the learner make that repair now and explain why. Then assess the relevant skill on a fresh, fully specified case.

Confirm it was not previously practised with feedback; solve and check the new case before assigning it. A follow-up design prompt is a starting brief, not automatically an equivalent test.

## Connect and extend learning

> Connect Repeating Efficiently to Helpful Classroom Sorter, adapt support and plan a later changed-case check.

### Helpful Classroom Sorter

Use a **loop** to **process** each item card in the test pile, with an explicit stop when no cards remain.

Keep the decision, **input**, result, explanation and remaining limitation in the project **evidence** record. Retrieve this record next week rather than restarting the project.

### Support and stretch

Use physical cards, spoken explanations and a three-stage drawing. Read aloud without giving the answer. Stretch: compare two **mechanism**s with the same visible **output** and explain what **evidence** distinguishes them.

### Exit reflection

Ask for one decision the learner can now justify and one situation requiring more **evidence** or human review. Confidence ratings are separate from correct independent performance.

### Bridge to the next week

Next: Week 12: Debug the Cause. Retrieve today's **evidence** and identify what carries forward and what is new.

### Support and fresh reassessment

Let students move a token from the test card to the **body** and back. Use written true and false labels as well as arrows. Provide a separate tally box for **body** executions.

Extension: explain why start 2, while not equal to 7 add 2 never reaches its reporting instruction with exact whole-number arithmetic.

W4 is secure when the learner traces the decrementing **loop**, includes zero **iteration**s, accepts 0 as a valid final result from 6 and explains why subtract 0 cannot end the run from 7.

After support, retry start 8, while above 2 subtract 3: tests 8, 5, 2; two **iteration**s and report 2. A correct final number without test reasoning is incomplete **evidence**.

Week 12 develops debugging from traces. Students will compare expected steps with observed steps, identify the first divergence and retest a repair.

### Plan the delayed return

Suggested return points in this level: ai-1/12, ai-1/14, ai-1/17. Use actual classroom dates, adjusting for holidays and gaps. One, three and six teaching weeks are scheduling choices, not a claimed optimal spacing.

At a delayed check, ask for the idea from memory and a changed application. Compare with the earlier independent response and record support used.

For a new level, revisit the previous capstone before checking the new prerequisites.

| Record | Teacher evidence |
|---|---|
| Learner code and date | |
| Concept and case used; prior exposure | |
| Explanation / trace / limits | |
| Support and first repair | |
| Changed case and delayed outcome | |
| Next teaching action | |

Use the **evidence** to choose reteaching or the next step. A completed page, confidence rating or attractive project does not by itself demonstrate understanding.

### Project evidence from this case

Give any repeated sorter step an explicit **stopping condition** and trace a zero-**iteration** example.

# Prepare, teach and assess

> Use this preparation summary alongside the detailed Repeating Efficiently lesson; it is a checklist rather than a replacement for the full sequence.

### Teaching target

Teaching target: Trace a **loop** by testing its condition before every repetition

Prepare the labelled worked-case picture, the matching workbook, fictional **input** cards and a place for individual responses. No paid AI account is required for the paper reasoning activities.

Session · Plan · Resources

> **Session:** Session 1 · Understand · 45 min
>
> **Plan:** 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.
>
> **Resources:** Use Your mission, Notice the evidence, See how it works, Compare the cases, Words that unlock the idea and Follow a worked example; record the brief recall in the workbook.

> **Session:** Session 2 · Practise · 45 min
>
> **Plan:** 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.
>
> **Resources:** Use Find and repair a mistake, Practise together, Run the investigation and Build your understanding; use the matching workbook W2 and misconception check.

> **Session:** Session 3 · Investigate and transfer · 45 min
>
> **Plan:** 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.

> **Resources:** Use Run the investigation, Connect your project, Show what you know, Study a complete case and Try a new case independently. Allow extra time for the full task set. Use a coding lab only when this week actually provides one; Your lesson route is the planning section.

Before class, solve the supported practice: Use the same below 8 **loop** with add 2. Make separate runs from 8 and 9. Explain how many times the **body** runs and what each run reports. Why would adding 2 once be an error?

### Supported practice answer

Reasoned practice answer: Both first tests are false. Each run has zero **iteration**s, reporting its unchanged starting value: 8 or 9. Adding 2 once would execute the **body** despite a false first test.

Check every learner before discussion: Use the worked case for **Loop**s and stopping rules. Proposed explanation: “Every **loop** runs at least once”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

### Repair the misconception

Misconception repair: Point to the test before the **body**. From 7 under below 7 the first test is false, so no update is permitted.

### Support without giving away the answer

Support: read the exact case aloud, enlarge the labelled picture, supply **input** cards and model only the first deciding step. Accept a spoken explanation or drawing; retain the reasoning goal.

### Extend the reasoning

Extension: Start at 10 instead of 2. Require a **prediction** and a reason before comparing. Expected result: The **loop** runs zero times and reports 10. Reason: Its first condition is already false, so no addition happens.

### Protect the independent attempt

Independent assessment: withhold the reserved answer until the learner has attempted the new case. Record support separately from correctness.

| Criterion | Not yet | With support | Independent |
|---|---|---|---|
| Explanation | Names an answer without a deciding fact | Uses a hint to link fact and decision | Explains the deciding fact in own words |
| Trace or test | Cannot connect input and result | Completes steps with prompts | Traces or tests a changed case accurately |
| Limits and revision | Claims more than evidence supports | Names a limit after a prompt | States a defensible limit and useful next test |

### Project checkpoint

Project checkpoint: Make a transparent first version. Keep a trace of how one **input** becomes an **output**; explain the rule or learned component.

Use the weekly teacher answer for topic-specific reasoning. A saved **progress** tick is an activity record; judge mastery from the learner's explanation and **evidence**.

## Use feedback, labs and repair paths

> Use feedback, earlier review and portfolios purposefully. Apply the coding routine only when this lesson has an executable lab.

Use the two formative questions as supported practice, not as independent mastery scores. Discuss the deciding **evidence** rather than accepting a guessed selection.

The feedback comes from the reviewed public worked case. Do not reveal the separately reserved independent teacher answer until the learner has attempted that case.

### For later coding lessons

> This week has no code block. Use this routine when a later lesson includes an executable lab.

Ask students to predict before running code. Preserve errors and actual **output**s; collect a changed test and an explanation. The branch-boundary and indexing lessons deliberately include failure cases.

Use the earlier worked-explanation links to address a specific missing building block, then return to the current question. Adapt this suggestion after observing the learner; it is not an automatic diagnosis.

Collect the completed-work portfolio as **evidence**. Record support separately from correctness, and assess explanation, trace/test and limitations. **Progress** ticks and quiz selections alone do not establish mastery.

---

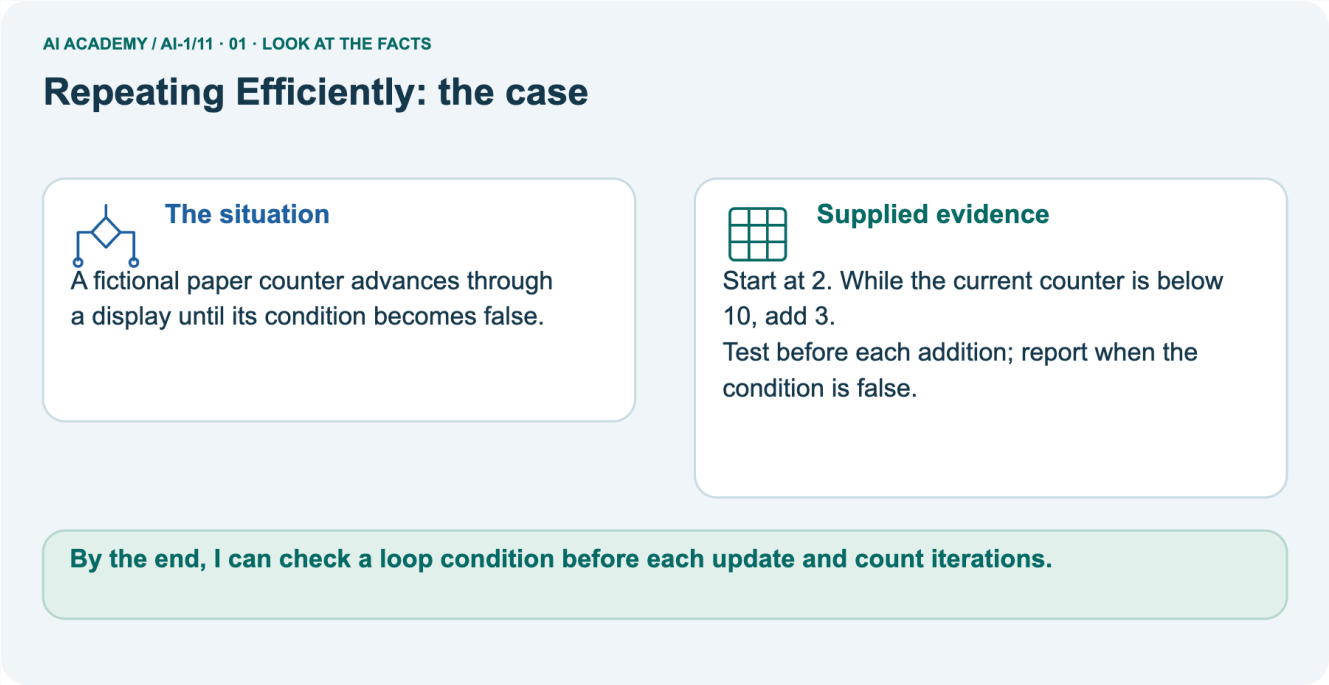


## Visual concept journey

> Show one Repeating Efficiently graphic at a time, request a prediction and discuss its trace before the independent assessment.

Use one picture at a time. Hide a worked result before asking for a **prediction**, then reveal it for feedback. These are supported teaching cases; retain the separate independent assessment for a fresh first attempt.

### 01. Repeating Efficiently: the case




Follow the labelled picture with the matching explanation. Open the vector diagram

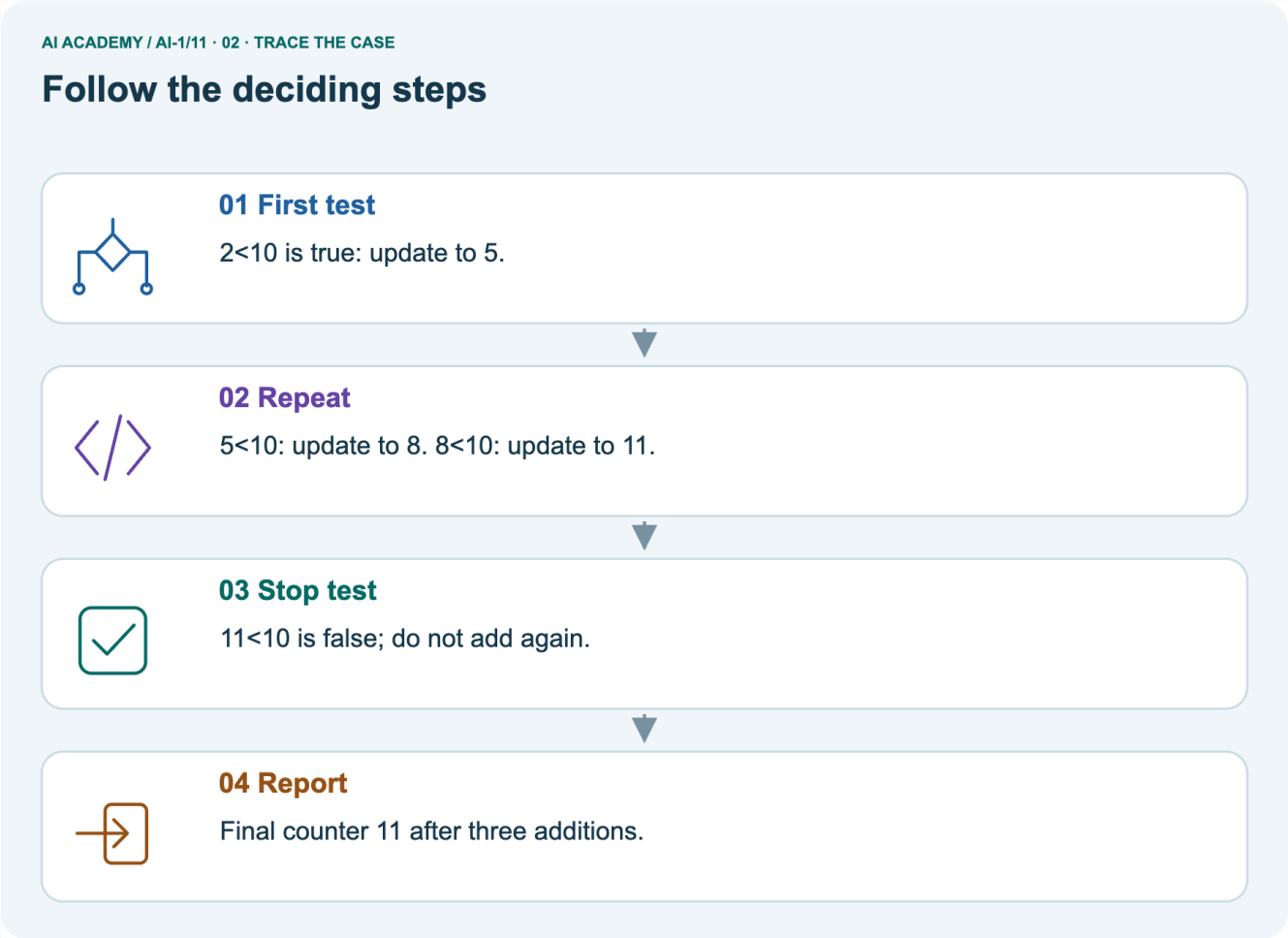
A fictional paper counter advances through a display until its condition becomes false. Supplied facts: Start at 2. While the current counter is below 10, add 3. Test before each addition; report when the condition is false.

> Point to the given fact that matters most. Explain why it matters.

#### Discuss while showing the picture

Teacher prompt: Point to the given fact that matters most. Explain why it matters. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

## 02. Follow the deciding steps




Follow the labelled picture with the matching explanation. Open the vector diagram

First test: 2<10 is true: update to 5. → Repeat: 5<10: update to 8. 8<10: update to 11. → Stop test: 11<10 is false; do not add again. → Report: Final counter 11 after three additions.

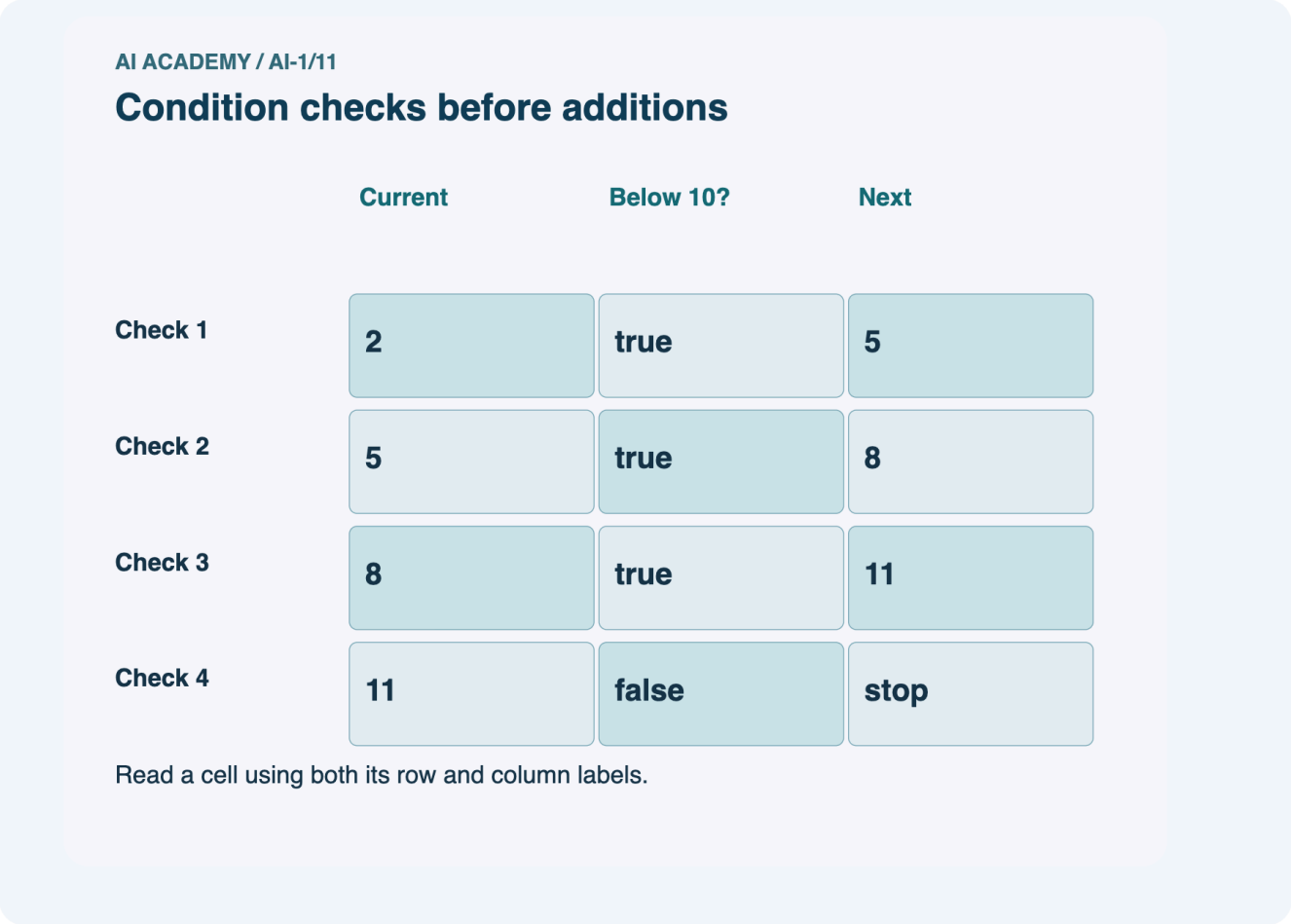
Point to each numbered stage and explain which recorded **input** it uses.

### Discuss while showing the picture

Teacher prompt: Point to each numbered stage and explain which recorded **input** it uses. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if

the image is difficult to see.

### 03. Condition checks before additions




Follow the labelled picture with the matching explanation. Open the vector diagram

Condition checks before additions. Check 1: Current = 2, Below 10? = true, Next = 5. Check 2: Current = 5, Below 10? = true, Next = 8. Check 3: Current = 8, Below 10? = true, Next = 11.

Check 4: Current = 11, Below 10? = false, Next = stop.

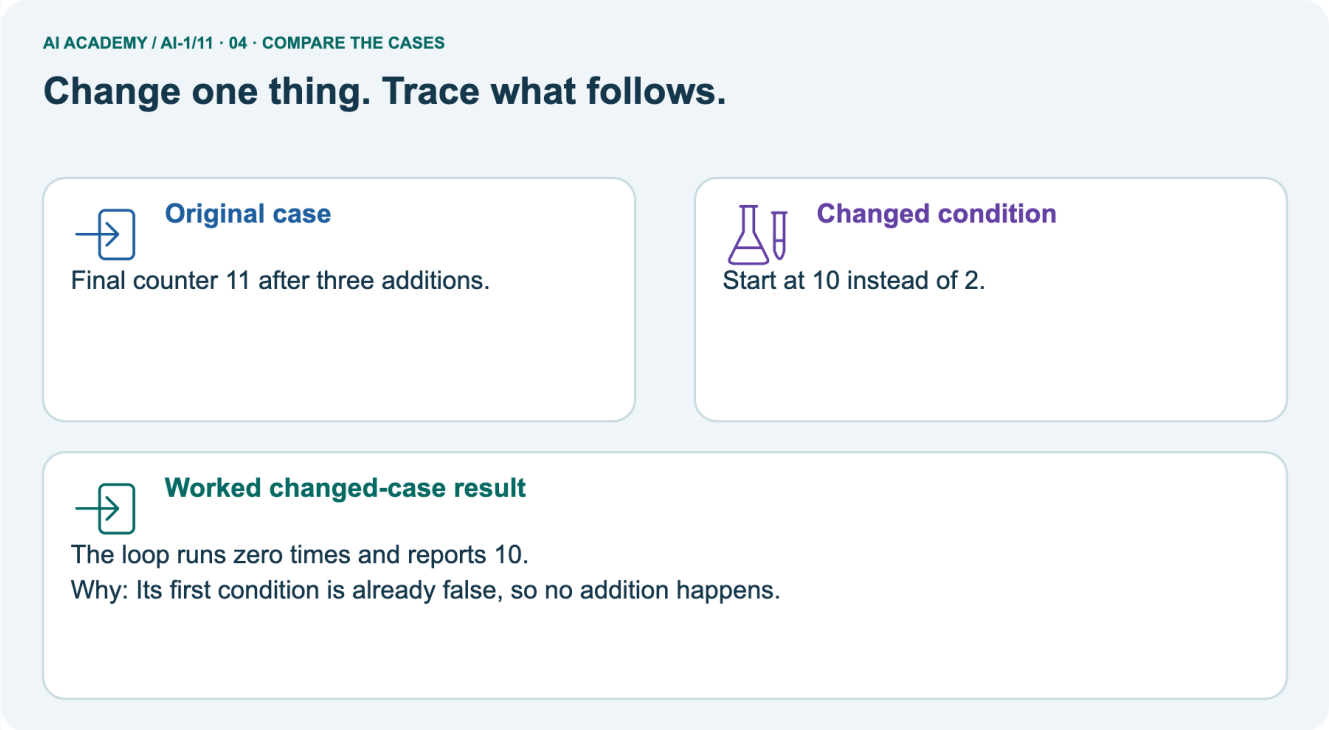
Read the labels and values. Explain the relationship the model shows.

### Discuss while showing the picture

Teacher prompt: Read the labels and values. Explain the relationship the model shows. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Check row/column meanings, units and denominators before discussing a difference. Supplied toy values are not real class or model results.

## 04. Change one thing. Trace what follows.




Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Final counter 11 after three additions. Changed condition: Start at 10 instead of 2. Expected result: The **loop** runs zero times and reports 10.

Reason: Its first condition is already false, so no addition happens.

Predict before checking the worked changed-case result. Name what changed and what the

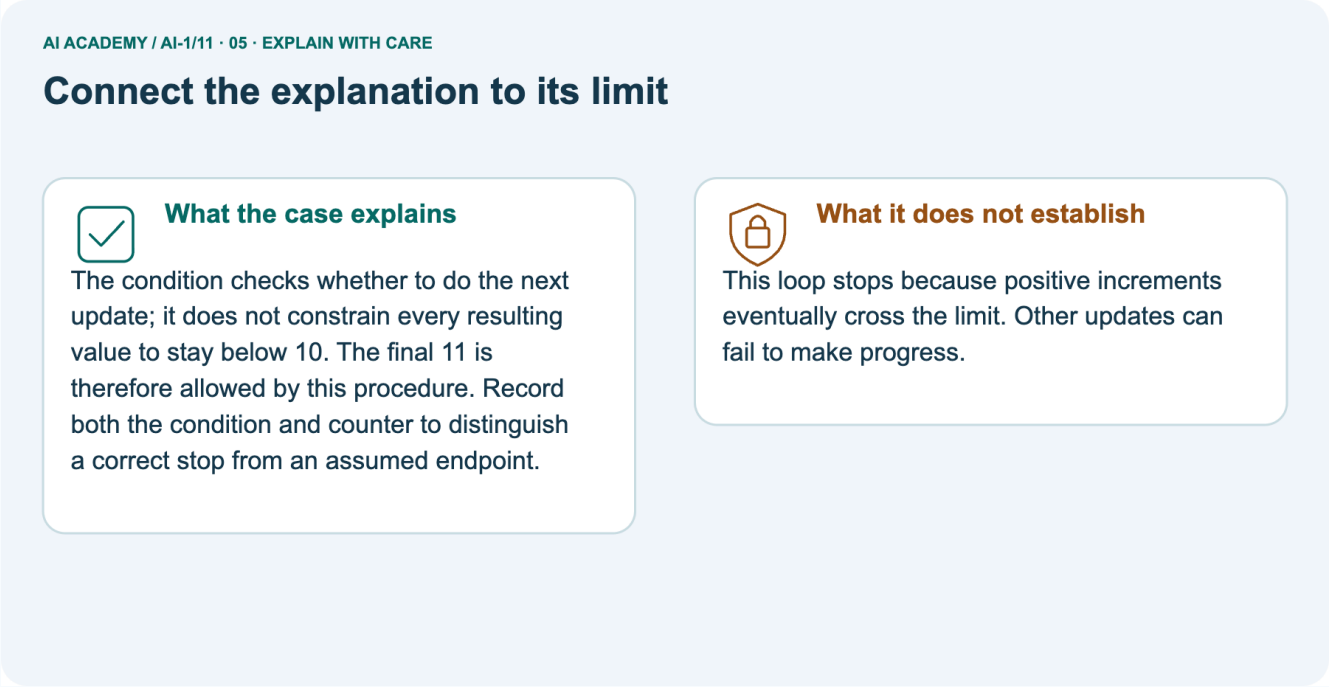
supplied facts allow you to conclude.

### Discuss while showing the picture

Teacher prompt: Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude. Ask learners to describe one label or arrow in ordinary words.

Use the adjacent transcript if the image is difficult to see. Changed-case reasoning: Its first condition is already false, so no addition happens.

### 05. Connect the explanation to its limit




Follow the labelled picture with the matching explanation. Open the vector diagram

The condition checks whether to do the next update; it does not constrain every resulting value to stay below 10. The final 11 is therefore allowed by this procedure.

Record both the condition and counter to distinguish a correct stop from an assumed endpoint. Limit: This **loop** stops because positive increments eventually cross the limit. Other updates can fail to make **progress**.

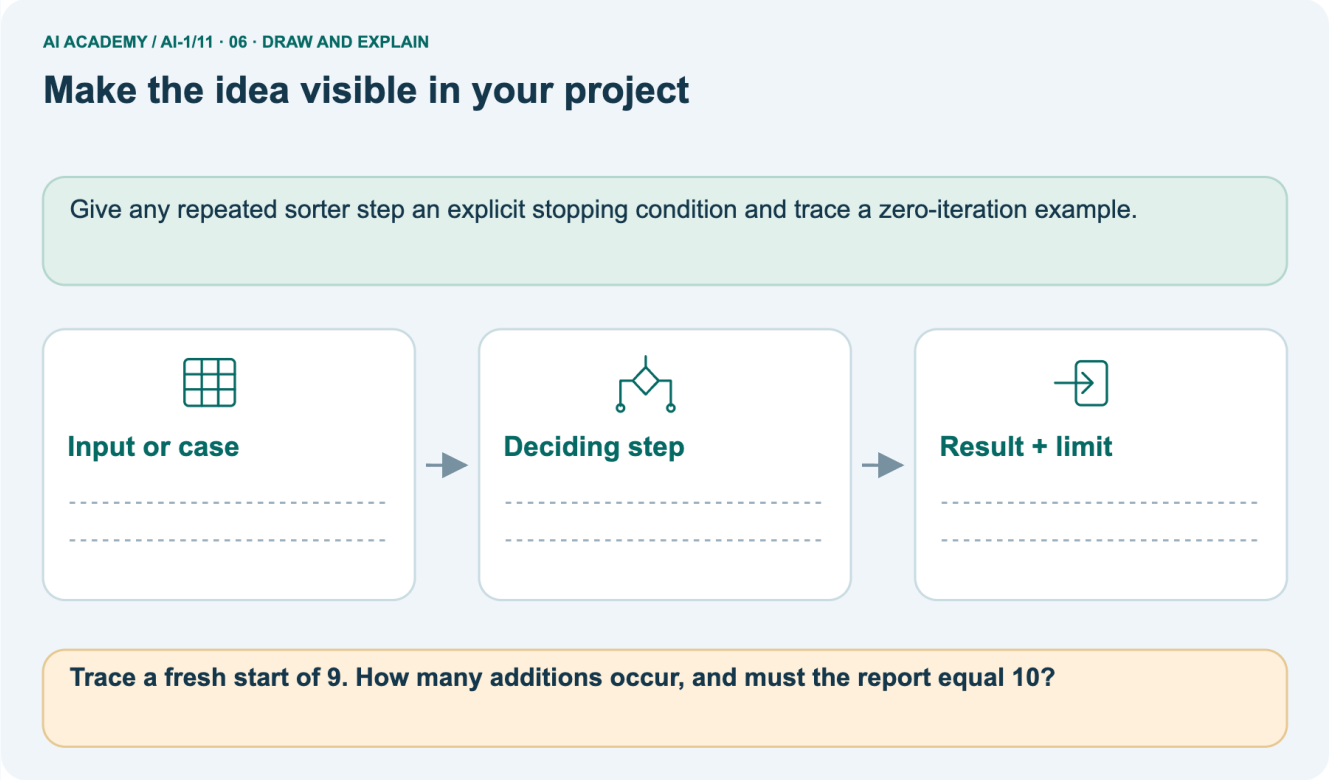
Explain the case, then point to a limit on the claim.

### Discuss while showing the picture

Teacher prompt: Explain the case, then point to a limit on the claim. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Keep the stated scope: This **loop** stops because positive increments eventually cross the limit. Other updates can fail to make **progress**.

### 06. Make the idea visible in your project




Follow the labelled picture with the matching explanation. Open the vector diagram

Give any repeated sorter step an explicit **stopping condition** and trace a zero-**iteration** example. Draw your **input**, deciding step, result and limit. Supported question: Trace a fresh start of 9.

How many additions occur, and must the report equal 10?

Draw your project trace and justify the deciding step using **evidence**.

### Discuss while showing the picture

Teacher prompt: Draw your project trace and justify the deciding step using **evidence**. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Supported practice key: 9<10 is true, so it becomes 12. Then 12<10 is false: one addition, report 12; equality with 10 is not required.

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.